D.3.1 ForeignKey

```
class ForeignKey(to, on_delete, limit_choices_to=None, related_name=None,
    related_query_name=None, to_field=None, db_constraint=True, **options)
```

This specifies a many-to-one relationship, pointing to the `Model` associated with this object. The `to` and `on_delete` arguments are required, with `to` specifying the `Model` of the related object. The `on_delete` argument specifies the behavior when the corresponding `Model` is deleted. The choices are `CASCADE`, which deletes the associated object; `PROTECT`, which prevents the deletion of the associated object; `RESTRICT`, which allows the deletion if there is another `ForeignKey` field on the same object that is also being deleted through a `CASCADE`; `SET_NULL`, which sets the value to NULL; `SET_DEFAULT`, which sets the field to its default; `SET(value)`, which sets the value to that given; and `DO_NOTHING`, which takes no action, possibly causing a database integrity error.

The `limit_choices_to` argument allows you to restrict the choices for this field during validation. It takes a dictionary, specifying field names and values for the corresponding `Model`. For example, `{"is_staff":True}` restricts the relationship to objects that have an `is_staff` field set to `True`. This argument can also be a callable that returns a dictionary.

The `related_name` argument specifies the name to be used as the backward relationship between the foreign object and this one. This allows you to override the default name—for example, `song_set` on an `Album Model` pointing to a `Song Model`. It can be set to `+` to stop a related name from being generated. Similarly, the `related_query_name` argument specifies the name of the attribute for the forward relationship.

The `to_field` argument indicates which field on the related object contains the value to store, indicating the relationship. It defaults to the ID field of the object. If given, the field must point to a primary key.

The `db_constraint` argument controls whether a constraint is created in the database for the inter-table relationship. It defaults to `True` and may need to be turned off for tables with legacy data not generated by Django or for database sharding.

D.3.2 ManyToManyField

```
class ManyToManyField(to, realted_name=None, related_query_name=None,
    limit_choices_to=None, symetrical=None, through=None,
    through_fields=None, db_constraint=True, db_table=None, **options)
```

This represents a many-to-many relationship between objects. It creates an extra table that maps the keys between related objects. The `to` argument is required, and it points to the related Model.

The `related_name`, `related_query_name`, `limit_choices_to`, and `db-constraint` arguments are all the same as `ForeignKey`.

When using a `ManyToManyField` to define a relationship between a `Model` and itself, the `symmetrical` argument indicates whether both the forward and backward relationship is put in the database. Setting it to `False` indicates a one-way relationship;